asr_text_correct

In-process Korean speech-to-text (fine-tuned Whisper large-v3) with Gemini transcript correction, plus near-real-time streaming.

GitHub: github.com/Aditi1612/asr_text_correct_library · Model: huggingface.co/fluffy16/whisper-ko-asr (gated) · v0.1.0

What it is

A Python library that runs the Korean ASR model **directly inside your application**, with no HTTP server and no calls to the pipeline API. You load the model once and call a function. Each transcript then goes to Google Gemini, which fixes misheard syllables, spacing and grammar.

```
audio → Whisper (fine-tuned, Korean) → transcript + confidence → Gemini → corrected_text
```

- **One-shot**: give it an audio file or waveform, and get back both the raw transcript and the corrected text.
- **Streaming**: push microphone frames continuously. Each time the speaker pauses, you get a corrected transcript of what they just said.
- **Either half alone**: STT only (no Gemini key needed), or correction only (no model download needed).

The decode and correction paths are copies of the production pipeline code. That covers the generation settings, text normalization, the repetition filter, the correction prompt, and Gemini key rotation. Transcription uses a fast greedy decode and needs about 4 GB of GPU memory.

Before you start: two kinds of access

What	Why	How to get it
Hugging Face access to the model	The model repo is **gated**. Only approved accounts can download the weights.	1. Sign in at huggingface.co and open fluffy16/whisper-ko-asr. 2. Click **Request access**. Skip this step if the maintainer has already added your account. 3. Wait for approval. 4. Create a **Read** token at huggingface.co/settings/tokens.
A Gemini API key	Correction calls the Google Gemini API.	Create a key at aistudio.google.com/apikey (free tier available), or ask the maintainer for one.

> Treat both the HF token and the Gemini key as passwords. Don't commit them to git or paste them into shared documents. Set them as environment variables instead (see below).

Requirements

Component	Notes
Python	3.9 or newer
`torch`, `torchaudio`	Install the build that matches your machine (CUDA or CPU) *first*. These are large and hardware-specific.
GPU	**Strongly recommended.** An NVIDIA GPU with at least 4 GB of free memory; the model runs in fp16. A CPU works but is much slower (see Performance).
RAM	About 8 GB free for loading the model.
Disk	About 6.2 GB for the model. It is downloaded once and cached in `~/.cache/huggingface`.
Network	Needed for the first model download and for every Gemini correction call.

Installation

```
# 1. install torch for your hardware (example: CUDA 12.1)
pip install torch torchaudio --index-url https://download.pytorch.org/whl/cu121
#    ...or CPU-only:
pip install torch torchaudio --index-url https://download.pytorch.org/whl/cpu

# 2. install the library (public GitHub repo)
pip install "asr-text-correct[hub] @ git+https://github.com/Aditi1612/
asr_text_correct_library.git"

# 3. credentials (put these in your shell profile or a .env your app loads)
huggingface-cli login                # paste your HF Read token, or: export HF_TOKEN=hf_...
export GEMINI_API_KEY=your-gemini-key
export ASR_HF_REPO=fluffy16/whisper-ko-asr   # optional: lets you omit hf_repo= in code
```

Optional extras:

Extra	Adds
`[hub]`	download the model from Hugging Face Hub (needed off the original server)
`[vad]`	`webrtcvad`, which detects speech more accurately than the built-in energy fallback
`[mic]`	`sounddevice`, only needed to run the microphone demo

Quick start: STT + correction

```
from asr_text_correct import ASRCorrector

pipe = ASRCorrector(hf_repo="fluffy16/whisper-ko-asr")
# ~6 GB download on first run, then cached

r = pipe.process("clip.wav")
r.transcript        # '나는 바지를 입고 단추를 채웁니다'    raw STT output
r.corrected_text    # '나는 바지를 입고 단추를 채웠다.'      after Gemini
r.confidence        # 0.899   STT confidence, 0..1
r.is_uncertain      # False   True if confidence < 0.15 or the transcript is empty
r.correction_ok     # True    False if Gemini failed and the fallback was used
r.timings           # {'stt': 0.2, 'text_correction': 0.77, 'total': 0.97}
r.to_dict()         # everything above as a plain dict (JSON-serializable)
```

`process()` accepts a file path, encoded bytes (`wav/flac/ogg`, plus `mp3/m4a` when ffmpeg is present), or a numpy / torch waveform passed with `input_sr=`.

Argument	Default	Meaning
`language`	`"korean"`	Whisper language; also accepts codes like `"KR"`, `"EN"`
`confidence_floor`	0.15	below this confidence, `is_uncertain` is set to `True`
`reference_text`	—	a ground-truth transcript; when given, `r.cer` (character error rate) is filled in
`prompt`	built-in	replaces the correction instructions for this call

Using only one half

```
from asr_text_correct import Transcriber, GeminiCorrector

# STT only: no Gemini key needed
asr = Transcriber(hf_repo="fluffy16/whisper-ko-asr")
asr.transcribe("clip.wav")                    # -> str
asr.transcribe("clip.wav", detailed=True)     # -> dict with confidence / is_uncertain

# correction only: no model download, just a Gemini key
fix = GeminiCorrector()
fix.correct("은 변원 앞에 있어요 약을 하우레 두 씩 드세요")
# -> '병원 앞에 있어요. 약을 하루에 두 시 씩 드세요.'

# combine objects you already have
pipe = ASRCorrector(transcriber=asr, corrector=fix)
```

Streaming (near real-time)

Your app captures audio from any source and pushes mono float32 chunks of any length. Each time the speaker pauses, the buffered utterance is transcribed, corrected and returned. There are no partial results; you get one complete transcript per utterance.

```
from asr_text_correct import ASRCorrector

pipe = ASRCorrector(hf_repo="fluffy16/whisper-ko-asr")
stream = pipe.stream(on_segment=lambda s: print(s.text, "->", s.corrected_text))

for chunk in your_mic_source():                       # e.g. 20-100 ms blocks, any sample
rate
    for seg in stream.accept(chunk, input_sr=48000):  # resampled internally if needed
        handle(seg.corrected_text, seg.start_time, seg.end_time)

stream.flush()                                        # finalize trailing speech when the
stream ends
```

- `accept()` returns a list of finished `Segment` objects. It is usually empty, and holds one segment when an utterance has just ended. Each segment has `.text` (raw STT), `.corrected_text`, `.start_time` and `.end_time`.
- `pipe.stream(correct=False)` skips Gemini, and `corrected_text` is then `None`.
- When an utterance ends, `accept()` blocks while STT and the Gemini call run, typically 1–2 s on a GPU. If your audio capture thread can't block, call `accept()` from a worker thread and feed it through a queue.
- A runnable microphone demo is in `examples/stream_mic.py`.

Tuning `pipe.stream(...)`

Argument	Default	Meaning
`end_silence_ms`	600	pause length that ends an utterance; this is roughly your perceived latency
`min_speech_ms`	250	utterances shorter than this are dropped as noise
`max_speech_ms`	15000	force-cut a segment that never pauses (Whisper's limit is 30 s)
`start_speech_frames`	3	consecutive speech frames needed to start capturing
`pre_roll_ms`	200	audio kept from just before speech starts, so the first syllable isn't clipped
`vad_aggressiveness`	2	webrtcvad strictness 0–3 (higher = more aggressive about calling things non-speech)

Gemini settings

Setting	How
API key	`export GEMINI_API_KEY=...` or `ASRCorrector(api_key="...")`
Several keys (rotation)	`export GEMINI_API_KEYS=key1,key2,key3` or `api_keys=[...]`. Requests rotate across the keys, and a rate-limited (429) or failing key is skipped automatically.
Model	`gemini-2.5-flash-lite` by default; change with `export GEMINI_MODEL=...` or `ASRCorrector(gemini_model="...")`
Custom instructions	`ASRCorrector(prompt="...")` or `process(..., prompt="...")`. The library appends the raw transcript after your text.

> **If Gemini fails** (a bad key, quota used up, or no network), you still get a result. `corrected_text` falls back to the raw transcript with extra spaces removed, and `correction_ok` is `False`, so check it if correction matters to you. If you would rather get an exception, use `GeminiCorrector(fallback=False)`.

Where the model comes from

Resolution priority: explicit path → Hugging Face repo → `ASR_MODEL_DIR` → local sibling checkout (original server only).

Source	How
Hugging Face Hub	`ASRCorrector(hf_repo="fluffy16/whisper-ko-asr")` or `export ASR_HF_REPO=fluffy16/whisper-ko-asr`
Local folder	`ASRCorrector("/path/to/checkpoint")` or `export ASR_MODEL_DIR=/path/to/checkpoint`
Pin a version	`ASRCorrector(hf_repo="fluffy16/whisper-ko-asr", hf_revision="<commit-sha>")`
Token in code	`ASRCorrector(hf_repo=..., hf_token="hf_...")` (an alternative to `huggingface-cli login`)

Performance (measured on the original server)

Times are for the STT call only, with the model already loaded. Gemini correction adds about 0.5–1 s per call, depending on the network.

Audio length	GPU (CUDA, fp16)	CPU (48 cores)
3.1 s	0.20 s	4.1 s
5.1 s	0.26 s	5.1 s
6.6 s	0.25 s	4.6 s

- **GPU**: about 0.2–0.3 s per short clip, using about 3.9 GB of GPU memory at peak (3.1 GB for the model itself).
- **CPU**: about 4–5 s per short clip, roughly 20× slower than the GPU. It's usable for occasional files, but too slow for live streaming.
- **Model load**: about 10 s from disk on the first start, and 1–5 s after that once the OS has the files cached.
- **First download from Hugging Face**: about 6.2 GB. The time depends on your network (about 50 s on the original server), and the files stay cached afterwards.
- These figures were measured on a shared server. A GPU that is busy with other jobs can be several times slower.

Limitations

- Correction is an AI rewrite. It can change verb endings or tense, add punctuation, and occasionally guess wrong. Keep `r.transcript` if you need the literal STT output.
- STT output is normalized: Hangul, digits and spaces only, with no punctuation. If the model gets stuck repeating itself, the transcript comes back empty (`""`) and `is_uncertain` is `True`.
- Like all Whisper models, it can “hear” words in pure noise or silence. On the sample set, real speech scored 0.88–0.91 confidence, while 3 s of noise produced a made-up word at 0.20. That is above the default 0.15 threshold, so it was *not* flagged. If your audio may contain no speech, raise the threshold, for example with `process(..., confidence_floor=0.5)`.
- The model transcribes whole utterances; there is no word-by-word live output. “Real-time” means the transcript appears shortly after each pause.
- One `ASRCorrector` runs one transcription at a time. For parallelism, use one instance per worker process.
- The model is Korean only, fine-tuned for dysarthric / atypical speech.
- Every correction sends the transcript text to Google's Gemini API. Make sure that is acceptable for your data.

Updating

When	Maintainer does	Users do
Library code changes	`git push` to the GitHub repo	`pip install --upgrade "asr-text-correct[hub] @ git+https://github.com/Aditi1612/asr_text_correct_library.git"`
Model checkpoint changes	re-runs `examples/upload_to_hf.py`	nothing; the next run pulls the new version (unless `hf_revision` is pinned)

Troubleshooting

Symptom	Fix
`GatedRepoError` or `401` when downloading	Your access hasn't been approved yet, or no token is set. Check the model page, then run `huggingface-cli login` or set `HF_TOKEN`.
`No Whisper checkpoint found`	pass `hf_repo=` / `model_dir=`, or set `ASR_HF_REPO`
`No Gemini API key found`	set `GEMINI_API_KEY`, or pass `api_key=`. For STT only, use `Transcriber`.
`correction_ok` is `False`	Gemini failed. Check your logs for the warning (invalid key, quota, network). The raw transcript was returned instead.
`Hugging Face Hub support needs 'huggingface_hub'`	install with the `[hub]` extra
VAD backend is `energy`, detection is rough	`pip install webrtcvad`
mp3 / m4a files fail to decode	install ffmpeg on the system
CUDA out of memory	free GPU memory, or use `ASRCorrector(..., force_cpu=True)` (slow)